SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2– Concept Mapping

Course Code:	CSA0313	**Course Title:**	Data Structures
CO Assessed:	CO1 – Imitation (BL3, BL4)	**Assessment:**	Assessment Tool 2 – Concept Mapping
Weightage:	10% of CIA	**Total Marks:**	100
CO1 Statement:	Basics, Data, Data object, Data structure, Abstract Data Types (ADT), realization of ADT in 'C', Primitive and non-primitive data structure, Linear and non-linear data structure Arrays & Recursion, Performance analysis and Measurement: Time and space analysis of algorithms, Average, best and worst-case analysis.		
Student Name:	S. SUSMITHA	**Reg. No.:**	192571068
Section / Batch:		**Date:**	14.07.2026

Code of Conduct

I (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Susmitha

Instructions

- This test contains 2 concept mapping. Total: 100 Marks.
- Evaluation of Answer Quality -Checking whether the answer includes: Concept Identification , Linkages, Organization, Integration of Literature, and Clarity
- No DTP printouts or electronic devices permitted. They should provide materials in PDF format.

Rubrics

Questions	Problem Understanding	Method Selection	Solution Process	Accuracy of Calculation	Interpretation of Results	Total Marks
Q1						
Q2						

NAME : S.SUSMITHA
REG.NO : 192571068
COURSE CODE : CSA0313

. 07. 2026 CO1 - AT 2

Q1. CONCEPT MAP - DATA ORGANIZATION IN SMART HEALTHCARE SYSTEM

DATA
Raw facts : temperature values, patient IDs, diagnostic codes

↓ IS INITIATED AS

DATA OBJECT
A named instance / value with type : eg. one "Patient Record" object

↳ IS ORGANIZED INTO

DATA STRUCTURE
organized collection of data objects ; array / list of patient records

ABSTRACT DATA TYPE (ADT)
Logical model : WHAT operations do, independent of HOW they're coded

↳ IS ABSTRACTED AS

OPERATIONS
insertPatient ( ), searchByID ( ), updateVitals ( ), deleteRecord ( )

→ DEFINES INTERFACE FOR ↓ CODED INSIDE

IMPLEMENTATION IN C
concrete realization : struct + pointers + functions operating on the struct

↳ IS REALIZED VIA

STRUCTURES (struct)
struct Patient (int id ; float temp ; char history [100] ;) ;

↳ GROUPS FIELDS USING

POINTERS
struct Patient * next ;
links records dynamically (linked list of patients)

ENABLES DYNAMIC LINKING VIA →

EXPLANATION

A smart healthcare system processes a large amount of patient information such as temperature readings, medical history and diagnostic details.
Proper organization of this data is essential for efficient storage, retrieval and processing.

1. DATA

↳ Raw facts & observations collected by the healthcare system.

↳ Eg: Temperature values, patient ID, medical history etc.

2. DATA OBJECT

↳ Meaningful collection of related data.

↳ Eg: PATIENT RECORD
↓
contains patient ID, medical history etc.

3. DATA STRUCTURE

↳ Systematically organizes multiple patient records.

Array → Fixed number of patient records stored.

Linked list → Dynamically stores & connects patient records.

4. ABSTRACT DATA TYPE (ADT)

↳ Defines what operations are performed, while hiding the implementation details.

↳ Eg:

- ✓ Insert patient
- ✓ search patient
- ✓ update patient vitals
- ✓ Delete patient record

5. IMPLEMENTATION IN C

↳ ADT is implemented using structures, pointers and functions.

↳ Eg:

```
struct Patient
{
  int id;
  float temperature;
  char history [100];
  struct Patient *next;
};
```

INTERPRETATION AND JUSTIFICATION

Data
↓
Data Object
↓
Data Structure
↓
ADT
↓
Implementation in C

This hierarchial organization provides:

- ✓ Efficient data storage
- ✓ Flexible record management
- ✓ Modularity
- ✓ Faster processing

(Q9) CONCEPT MAP : ARRAY → LINKED LIST → STACK → QUEUE → OPERATIONS

MUSIC STREAMING PLAYLIST MANAGEMENT

ARRAY

Fixed-size, contiguous storage of songs, fast index-based access

↓

LINKED LIST

Dynamic nodes (song + next pointer); easy insert/remove mid-playlist

↓

STACK (LIFO)

Built on array/linked list; access restricted to one end (top)

→ **PLAYLIST USE : "Previous/Undo"**

Last song played is popped first → backtrack through history

↓

QUEUE (FIFO)

Built on array/linked list; access restricted to front (dequeue) & rear (enqueue)

→ **PLAYLIST USE : "Up Next"**

Songs play in the order added → sequential auto play queue

↓

OPERATIONS

Insert (add song), Delete (remove song), Traverse (browse playlist)

→ **MOST SUITABLE OVERALL**

Linked list : O(1) insert/delete without shifting elements

EXPLANATION

1. Array
   - ↳ Stores songs in contiguous memory.
   - ↳ Direct access : O(1)
   - ↳ Insert / Delete : O(n) [Shifting required]

2. Linked List
   - ↳ Stores songs as linked nodes.

[Song A | Next] → [Song B | Next] → [Song C | NULL]

   - ↳ Insert / Delete : O(1) (position known)
   - ↳ Search : O(n)

3. Stack (LIFO)
   - ↳ Used for playback history
   - ↳ Operations : Push, Pop, Peek
   - ↳ Push / Pop : O(1)

4. Queue (FIFO)
   - ↳ Used for Up Next Playlist
   - ↳ Operations: Enqueue, Dequeue
   - ↳ Enqueue / Dequeue: O(1)

5. Playlist Operations
   - ↳ Insert - Add song
   - ↳ Delete - Remove song
   - ↳ Traverse - Display / Search / Play songs